Module 12

Tahalil Morsilin

April 20, 2026

1. 8-Bit Adder

- **Implementation:** Constructed 8 individual Full Adder subcircuits. Split the 8-bit inputs to feed each Full Adder individually, and chained the `Cout` (Carry Out) of bit n into the `Cin` (Carry In) of bit $n + 1$ to ripple the carry down the line.
- **Testing:**
  - *Test Case 1 (Basic Addition):* `0x01` + `0x02` = `0x03`
  - *Test Case 2 (Ripple):* `0x0F` + `0x01` = `0x10`
  - *Test Case 3 (Overflow):* `0xFF` + `0x01` = `0x00` (Carry = 1)

2. Adder Using a Decoder

- **Implementation:** First created a `Decoder1Bit` subcircuit. Used a 3-to-8 Decoder to map the 3 inputs (A, B, Cin) to minterms. Routed minterms 1, 2, 4, 7 through an OR gate to output the `Sum`, and minterms 3, 5, 6, 7 through an OR gate to output `Cout`. Chained 4 of these subcircuits together.
- **Testing:**
  - *Test Case 1 (Basic Addition):* `0x2` + `0x1` = `0x3`

– *Test Case 2 (Overflow):* `0xF` + `0x1` = `0x0` (Carry = 1)

3. 8-Bit Multiplexer (4 Inputs)

- **Implementation:** Utilized a single 4-to-1 Multiplexer (Select Bits = 2, Data Bits = 8). Wired four hex constants (`0x08`, `0x10`, `0x20`, `0x40`) directly into data pins 0 through 3.
- **Testing:**
  - Cycled the `Selector` pin from `00` to `11` to verify the output correctly stepped through $8 \rightarrow 16 \rightarrow 32 \rightarrow 64$.

4. 16-to-1, 8-Bit Multiplexer

- **Implementation:** Arranged five 4-to-1 multiplexers in a two-layer hierarchy. Layer 1 consists of four multiplexers that intake the 16 constants, controlled by select bits `0-1`. The outputs of Layer 1 feed into a single Layer 2 multiplexer, which is controlled by select bits `2-3` to funnel the data into a single 8-bit output.
- **Testing:**
  - *Test Case 1:* Selector `0000` = Output Constant 1 (`0x01`)
  - *Test Case 2:* Selector `0100` = Output Constant 5 (`0x05`)
  - *Test Case 3:* Selector `1111` = Output Constant 16 (`0x10`)